```
class TruffleDecorator extends PizzaDecorator
{
    public function getDescription(): string
    {
        return $this->decoratedPizza->getDescription() . ', with truffle
        shavings';
    }

    public function getCost(): float
    {
        return $this->decoratedPizza->getCost() + 3.50;
    }
}
```

All we need to do now is order some pizza!

```
$orderOne = new BlueCheeseDecorator(new Pizza());
$orderTwo = new TruffleDecorator(new BlueCheeseDecorator(new Pizza()));

print $orderOne->getDescription() . '. Costs ' . $orderOne->getCost() .
PHP_EOL;
print $orderTwo->getDescription() . '. Costs ' . $orderTwo->getCost() .
PHP_EOL;

Output:
Our famous pizza, with blue cheese. Costs 13.49
Our famous pizza, with blue cheese, with truffle shavings. Costs 16.99
```

As you can see, the description and costs are obtained from the outermost decorator objects, assembled as they delegate to their wrapped object(s). Note that we've instantiated all the required objects in one statement; a real application is, of course, free to add decorators at any point, no doubt using some other (creational) design pattern: good patterns will play nicely with other patterns.

This is just a taster (sorry) of structural patterns; there are others:

- Adapter (allows collaboration between objects with incompatible interfaces)
- Bridge (splits large sets of components into separate hierarchies)